Adding a Voltage Ratio Reading

This guide assumes you've been through `1 DigitalInput.md` and have a working file that changes a circle when a digital input button is pressed.

We're going to swap out our DigitalInput channel for a **VoltageRatio** channel - a continuous value roughly between 0 and 1, which is what Phidget's analog sensors, like the slider, report.

Step 1 — Add CSS for the ratio visual

This time we are starting with *style* ...

To represent the voltage ratio value, we are going to add a progress that changes in response to a user moving the slider. Your `<style>` block already has a `/* Button visual */` section for `#buttonVisual`. Add a new `/* Voltage ratio visual */` section after it:

```
/* Voltage ratio visual */
#ratioVisual {
  margin: 32px auto;
  max-width: 480px;
}

#ratioValue {
  font-size: 2.4rem;
  font-weight: bold;
  text-align: center;
  font-family: "Courier New", monospace;
}

#ratioBarTrack {
  width: 100%;
  height: 24px;
  background: #d9dce1;
  border-radius: 12px;
  overflow: hidden;
  margin-top: 12px;
}

#ratioBarFill {
  height: 100%;
  width: 0%;
  background: #3a9d4f;
  transition: width 0.1s ease;
}
```

Step 2 — Add markup for the ratio visual

In the body, you already have a section for the button:

```
<section>
  <h2>Button State</h2>
  <div id="buttonVisual" class="neutral">NEUTRAL</div>
</section>
```

Add a new section right after it:

```
<!-- The voltage ratio visual -->
<section>
  <h2>Voltage Ratio</h2>
  <div id="ratioVisual">
    <div id="ratioValue">--</div>
    <div id="ratioBarTrack">
      <div id="ratioBarFill"></div>
    </div>
  </div>
</section>
```

There's no `pressed` / `neutral`-style class here — a voltage ratio doesn't have two discrete states, so there's nothing to toggle. Instead there are two elements JavaScript will reach into separately: `#ratioValue` (the number) and `#ratioBarFill` (the bar's width). `--` is a placeholder shown before we've received a real reading, matching the "Waiting for connection..." placeholder already used in the raw state panel.

Step 3 — Add element references

In the script block, you already have a line for the button visual:

```
const buttonVisual = document.getElementById("buttonVisual");
```

Add two new lines beneath it for the elements from Step 2:

```
const ratioValue = document.getElementById("ratioValue");
const ratioBarFill = document.getElementById("ratioBarFill");
```

Step 4 — Add a second Phidget object variable

You already have:

```
let digitalInput;
```

Add a second `let` declaration for the new channel, right after it:

```
let voltageRatioInput;
```

Both channels will be created, opened, and closed independently, so each gets its own variable to hold onto the object between callbacks.

Step 5 — Add a display helper for the ratio

You already have `setButtonVisual`. Add a new function after it:

```
// Voltage ratios from this sensor type range roughly 0 - 1
function setRatioVisual(ratio) {
  if (ratio == null) {
    ratioValue.textContent = "--";
    ratioBarFill.style.width = "0%";
  }
  else {
    ratioValue.textContent = ratio.toFixed(4);
    ratioBarFill.style.width = `${ratio * 100}%`;
  }
}
```

This follows the same pattern as `setButtonVisual` — it just takes a number instead of a boolean. A few details worth understanding:

- **The `null` guard clause.** `setButtonVisual` never needed this because a button's state is always `true` or `false`. This function needs to handle "no reading yet" as a distinct case, since we'll call `setRatioVisual(null)` on disconnect (Step 8) to reset the display.
- **`ratio.toFixed(4)`** formats the number to 4 decimal places (e.g. `0.5731`) so the display doesn't jump around with long floating-point noise like `0.57309999997`.

Step 6 — Add the voltage ratio channel inside `connectToPhidgetServer`

Inside `connectToPhidgetServer`, you already open the digital input channel:

For now, comment that method out (using `//`). We can connect multiple channels at time, but we only have 1 wire available for our demo.

Add a method to open a Voltage Ratio input, and call that method inside connectToPhidgetServer:

```
async function connectToPhidgetServer(address, port) {
  ...

  await connection.connect();

  // await openDigitalInput();
  await openVoltageRatioInput();

  setConnectedUI(true);
}

async function openDigitalInput() {
  ...
}

async function openVoltageRatioInput() {
  voltageRatioInput = new phidget22.VoltageRatioInput();
```

```
  voltageRatioInput.isHubPortDevice = true;
  voltageRatioInput.hubPort = 0;

  voltageRatioInput.onAttach = () => {
    updateRawStatePanel({
      status: "Attached",
      deviceName: voltageRatioInput.deviceName,
      serialNumber: voltageRatioInput.deviceSerialNumber,
      channel: voltageRatioInput.channel,
      voltageRatio: voltageRatioInput.voltageRatio
    });
    setRatioVisual(voltageRatioInput.voltageRatio);
  };

  voltageRatioInput.onDetach = () => {
    updateRawStatePanel({ status: "Device detached" });
    setRatioVisual(null);
  };

  voltageRatioInput.onVoltageRatioChange = (voltageRatio) => {
    setRatioVisual(voltageRatio);
    updateRawStatePanel({
      status: "Voltage ratio change",
      deviceName: voltageRatioInput.deviceName,
      serialNumber: voltageRatioInput.deviceSerialNumber,
      channel: voltageRatioInput.channel,
      voltageRatio: voltageRatio,
      timestamp: new Date().toISOString()
    });
  };

  await voltageRatioInput.open(5000);
}
```

A few things worth noticing:

- `phidget22.VoltageRatioInput()` is a different channel class from `phidget22.DigitalInput()` — this is the line that tells the library what kind of hardware to look for. `isHubPortDevice = true` and `hubPort = 0` mean the same thing here as they did for the digital input: "the channel plugged into port 0.". If we wanted to connect multiple sensors at the same time, we'd need to update the hubPort to match which port each sensor is actually connected too.
- `voltageRatioInput.voltageRatio` is the property this channel type exposes for its current reading, the same way `digitalInput.state` was the property for the button. It's read once in `onAttach` (in case the sensor already has a value the moment we connect) and passed along on every `onVoltageRatioChange`.
- **Note there's no `!` inversion here**, unlike `setButtonVisual(!state)` for the button. That inversion was a workaround for that particular sensor. A voltage ratio is a continuous physical measurement, so `setRatioVisual(voltageRatio)` passes the raw value straight through.

Step 7 — Add cleanup for the voltage ratio channel on disconnect

You already have:

```
async function disconnectFromPhidgetServer() {
  if (digitalInput) {
    try {
      await digitalInput.close();
    } catch (error) {
      // Channel may already be closed — nothing to do here.
    }
    digitalInput = null;
  }
  setButtonVisual(false);
  setConnectedUI(false);
  updateRawStatePanel({ status: "Disconnected" });
}
```

Add a matching block for the voltage ratio channel, and a call to reset its visual, without touching the existing lines:

```
async function disconnectFromPhidgetServer() {
  if (digitalInput) {
    try {
      await digitalInput.close();
    } catch (error) {
      // Channel may already be closed — nothing to do here.
    }
    digitalInput = null;
  }
  if (voltageRatioInput) {
    try {
      await voltageRatioInput.close();
    } catch (error) {
      console.warn(error);
      // Channel may already be closed — nothing to do here.
    }
    voltageRatioInput = null;
  }
  setButtonVisual(false);
  setRatioVisual(null);
  setConnectedUI(false);
  updateRawStatePanel({ status: "Disconnected" });
}
```

Each channel gets its own guarded close, since either one could already be closed or never have successfully opened. Both display resets (`setButtonVisual(false)` and `setRatioVisual(null)`) run before the final status update, so the page returns to a fully neutral state regardless of which channels were active.

Trying it yourself

1. Make sure a Phidget Network Server is running and reachable, with a voltage ratio sensor (the slider sensor) plugged into hub port 0.
2. Open your updated file in a browser.
3. Enter the server's address and port (default `localhost:8989`) and click Connect.

4. Watch the raw state panel for `"Attached"` messages as each channel comes online. Move the sensor — the number and bar should update.

Bonus Round

Again, let's make this more fun. Try adding css `transform` styles to an image to reflect where the slider is.